AMATH 482/582: HOMEWORK 5

HYO JUNG YOON

University of Washington, Seattle, WA
hyoon94@uw.edu

Abstract. Spectral clustering and semi-supervised learning are applied to the 1984 U.S. House voting records dataset in this homework. Each congress member is represented by a voting vector across 16 bills, and similarities between members are used to construct a weighted graph. The graph Laplacian is used to perform spectral clustering, where the Fiedler vector separates the members into two groups corresponding to political parties. The effect of the similarity parameter σ on clustering accuracy is analyzed. In the second part, Laplacian eigenvectors are used as features in a semi-supervised regression model to predict party labels when only a small number of labeled examples are available.

1. Introduction and Overview

Understanding group structure in data is an important problem in machine learning and data analysis. In this homework, we study how spectral clustering and semi-supervised learning can be used to classify politicians based on their voting behavior. The dataset consists of voting records of 435 members of the U.S. House of Representatives across 16 bills. Each voting record is converted into numerical form so that similarities between members can be measured.

First, we construct a similarity graph based on voting patterns and use the graph Laplacian to perform spectral clustering. The second eigenvector of the Laplacian, known as the Fiedler vector, is used to separate the politicians into two clusters corresponding to the two major political parties. We then study how the similarity parameter σ affects clustering accuracy.

In the second part of the report, we use Laplacian eigenvectors as features in a semi-supervised learning setting. Only a small number of party labels are assumed to be known, and a regression model is trained to predict the remaining labels. The results show that the structure of the similarity graph allows the algorithm to achieve high classification accuracy even when only a limited number of labeled examples are available.

2. Theoretical Background

Our goal in this homework is to determine which politicians belong to which political party based only on how they vote on different bills. At first, we ignore the party labels and try to uncover the group structure directly from the voting patterns.

We start with a raw dataset which contains 435 politicians and 16 votes. Each politician is represented by a voting record across the 16 bills. For the algorithm, we convert the votes into numbers: yes $\to +1$, no $\to -1$, and unknown $\to 0$. This means each politician becomes a vector in $\mathbb{R}^{16}$. Writing these row vectors as $x_1, x_2, \ldots, x_{435}$ gives the input matrix

$$X \in \mathbb{R}^{435\times 16}.$$

Now we can compare voting patterns mathematically.

Date: March 13, 2026.

Next we ask how similar two politicians are based on their voting records. If two politicians vote the same way most of the time, they are considered very similar. If they often vote in opposite ways, they are considered very different. Using this idea, we compute a similarity score between every pair of politicians. The similarity score is defined by the Gaussian weight function

$$W_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right),$$

where x_i and x_j are the voting vectors of two politicians and σ controls the similarity scale. If two voting vectors are close together, then W_{ij} is close to 1. If they are far apart, then W_{ij} becomes much smaller. All of these pairwise similarities form the similarity matrix W.

Now we turn this into a social network graph. Each politician becomes a node, and we draw connections between them based on how similar their voting patterns are. Politicians who vote similarly will be strongly connected, while politicians who vote differently will be weakly connected. This gives a weighted graph that represents the voting network.

Next we compute something called the degree. This tells us how strongly a politician is connected to everyone else overall. If a politician votes similarly to many others, they will have a large degree. If they vote very differently from most others, they will have a small degree. These values form the degree matrix D, where each diagonal entry is the total connection strength of one node:

$$D_{ii} = \sum_{j=1}^{n} W_{ij}.$$

Using the similarity matrix W and the degree matrix D, we define the unnormalized graph Laplacian

$$L = D - W.$$

The Laplacian matrix encodes the overall network structure. Instead of analyzing the graph visually, we study it using linear algebra.

We then split the politicians into two groups using the structure of the similarity network. This is done without using the party labels, and instead relies only on how the politicians are connected in the graph. Inside each group, the connections are strong, while the connections between the groups are weak. Because of this, the graph naturally separates into two communities, and spectral clustering identifies this split.

To do this, we compute the eigenvalues and eigenvectors of the Laplacian matrix:

$$Lq_k = \lambda_k q_k,$$

where the eigenvalues are ordered as

$$0 = \lambda_0 \leq \lambda_1 \leq \lambda_2 \leq \cdots.$$

The first eigenvector corresponds to the zero eigenvalue. The second eigenvector q_1, called the Fiedler vector, is the one used for clustering. You can think of the eigenvectors as patterns hidden within the graph. One of these patterns reveals how the network naturally separates into two communities.

When we compute the second eigenvector, every politician gets a number. Politicians in one group tend to have positive values, while politicians in the other group tend to have negative values. This happens because the Fiedler vector is trying to separate the graph into two groups with weak connections between them. We then turn this into a classifier by splitting according to sign:

$$\hat{y} = \text{sign}(q_1).$$

Positive values are assigned to one cluster and negative values are assigned to the other. Since this is an unsupervised problem, the labels may be flipped relative to the true party labels, so the sign may need to be reversed when computing accuracy.

Once the clustering is done, we compare the predicted groups to the actual party labels. If most politicians are placed in the correct group, then the clustering accuracy is high. The homework asks us to vary the parameter σ to see which similarity scale gives the best clustering performance.

In the second part, we pretend that only a small number of party labels are known. Instead of guessing blindly, we use the spectral features from the Laplacian as new data features. The homework defines the Laplacian embedding

$$F(x_j) = \big((q_0)_j, (q_1)_j, \ldots, (q_{M-1})_j\big) \in \mathbb{R}^M,$$

where $q_0, q_1, \ldots, q_{M-1}$ are the first M eigenvectors of the Laplacian. Stacking these feature vectors for all politicians gives the embedding matrix

$$F(X) \in \mathbb{R}^{435\times M}.$$

If we only know the labels of the first J politicians, then we form a matrix $A \in \mathbb{R}^{J\times M}$ from the first J rows of $F(X)$ and a vector $b \in \mathbb{R}^J$ from the first J entries of the true label vector y. We then fit a simple regression model using least squares:

$$\hat{\beta} = \arg\min_{\beta} \|A\beta - b\|^2.$$

After that, we use the fitted model to predict labels for all politicians:

$$\hat{y} = \operatorname{sign}(F(X)\hat{\beta}).$$

Because the spectral features already capture the structure of the voting network, the regression model can correctly classify many of the unlabeled politicians. In this way, the graph structure helps us extend a small number of known labels to the rest of the dataset.

3. Algorithm Implementation and Development

All computations for this homework were implemented in Python using Google Colab. The dataset was first preprocessed by converting the categorical voting records into numerical values, where "yes" votes were mapped to $+1$, "no" votes to -1, and unknown votes to 0.

The similarity matrix and graph Laplacian were constructed using standard numerical operations with the NumPy library. Eigenvalues and eigenvectors of the Laplacian were computed using linear algebra routines from SciPy. Spectral clustering was performed by extracting the Fiedler vector and classifying members according to the sign of its entries. In the semi-supervised learning step, a least-squares regression model was implemented to learn a mapping from the Laplacian embedding features to the party labels.

Plots such as the clustering accuracy versus σ were generated using the matplotlib library.

4. Computational Results

I first converted the categorical data into numbers in Google Colab since machine learning algorithms (like spectral clustering and regression) require numeric inputs. This step prepares the data so it can be used to construct the graph Laplacian and apply spectral clustering.

Spectral Clustering

I computed all the pairwise squared distances between members based on how they voted on the 16 vote vectors. The matrix W turns these distances into similarities. If two members vote similarly, their distance is small and their weight is close to 1. If they vote differently, their weight is closer to 0. The matrix D stores the row sums of W. Therefore, $L = D - W$ builds the unnormalized Laplacian.

I then computed the eigenvalues and eigenvectors. The homework asks for the second eigenvector since the first eigenvector corresponds to the zero eigenvalue. The second one is the Fiedler vector, which will be used for clustering. It satisfies

$$Lq_1 = \lambda_1 q_1.$$

The Laplacian matrix $L \in \mathbb{R}^{435\times435}$ was constructed from the similarity graph. The smallest eigenvalue was approximately 6.14×10^{-18}, which is effectively zero as expected for a graph Laplacian. The next few eigenvalues were 3.74×10^{-3}, 5.58×10^{-3}, 1.05×10^{-2}, and 1.36×10^{-2}.

As expected, the smallest eigenvalue of the Laplacian was approximately zero and its eigenvector was constant. The second eigenvector (the Fiedler vector) captured the main two-cluster structure of the graph.

Next, I convert the Fiedler vector into two classes by applying the sign function. Negative values are assigned a class label of -1 and positive values are assigned a class label of $+1$. Since spectral clustering does not guarantee which cluster corresponds to which label, I also check the sign-flipped version of the predictions and choose the orientation that gives the higher classification accuracy.

Using this approach, the clustering accuracy was 87.36%, with 55 members misclassified out of 435.

I then varied the parameter σ in the Gaussian weight function and measured how it affected clustering accuracy. The parameter σ controls the scale of similarity in the graph. For each value of σ in the range $(0, 4]$, the similarity matrix W was constructed, the Laplacian $L = D - W$ was computed, the Fiedler vector was extracted, and clustering was performed using $\text{sign}(q_1)$. The classification accuracy was then measured.

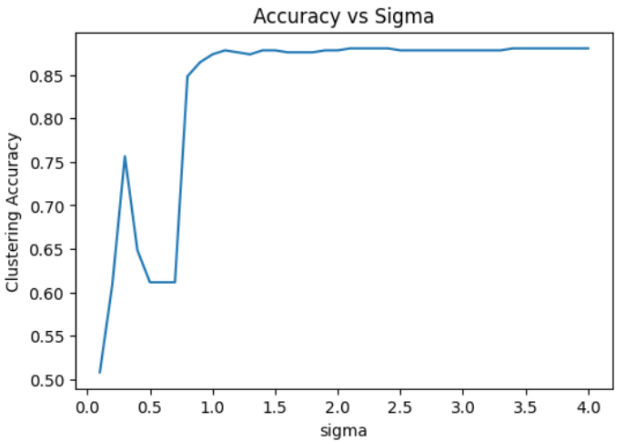

FIGURE 1. Clustering accuracy as a function of the Gaussian similarity parameter σ.

The results are shown in Figure 1. The optimal value was $\sigma^* \approx 2.1$, which produced the highest clustering accuracy of about 88.05%.

The plot shows three main behaviors. For small σ values (0–0.6), the accuracy is low (around 50%–75%). In this case, the similarity graph becomes too sparse because only nearly identical voting records are considered similar. As a result, the graph breaks into many small pieces and the algorithm struggles to detect the larger party structure.

For moderate values of σ (around 0.8–2.2), the accuracy increases to about 85%–88%. In this range the similarity graph better captures the overall structure of the voting patterns, allowing the algorithm to separate the two parties more effectively.

For large σ values (≥ 2.5), the accuracy remains roughly constant at around 88%. In this case most members are connected in the similarity graph, but Democrats still tend to be slightly more

similar to Democrats and Republicans to Republicans. Because the voting data already contains a strong underlying party structure, the clustering performance remains stable even when σ becomes large.

The homework defines the embedding

$$F(x_j) = (q_0(j), q_1(j), \ldots, q_{M-1}(j)),$$

where q_i are the eigenvectors of the Laplacian.

Each congress member is represented by a vector of Laplacian eigenvector values. These eigenvectors act as features that describe the structure of the similarity graph.

Using $M = 3$, this produces a feature matrix $F(X) \in \mathbb{R}^{435\times 3}$, where each row corresponds to one congress member and each column corresponds to one Laplacian eigenvector.

Instead of representing each member by their original 16 voting features, the Laplacian embedding represents each member using three graph-based coordinates that capture the overall structure of the voting similarity network.

Next, we assume that the party labels are known for only a small subset of congress members. However, the voting patterns of all members are still available. By using the Laplacian embedding from the previous step, the labeled members can be used to train a model that predicts the party labels of the remaining members.

Let J denote the number of labeled congress members. The first J rows of the Laplacian embedding matrix $F(X)$ form the matrix $A \in \mathbb{R}^{J\times M}$, and the corresponding party labels form the vector $b \in \mathbb{R}^J$.

In our implementation with $M = 3$ and $J = 10$, this produced

$$A \in \mathbb{R}^{10\times 3}, \quad b \in \mathbb{R}^{10}.$$

Semi-Supervised Regression

Next, a linear regression model was trained using the labeled members and then used to predict party labels for all 435 members.

The coefficients were obtained by solving the least squares problem

$$\hat{\beta} = \arg\min_{\beta} \|A\beta - b\|^2.$$

This finds the coefficients β that best map the Laplacian graph features to the party labels using the J labeled members. The learned model was then applied to all members by computing

$$\hat{y} = \text{sign}(F(X)\hat{\beta}).$$

Using $J = 10$ labeled members, the model misclassified 54 out of 435 members, corresponding to a semi-supervised learning accuracy of 87.59%.

This result shows that even with only a small number of labeled examples, the algorithm can achieve good predictions by using the structure of the similarity graph.

Here, A contains the Laplacian features of the labeled members and b contains their corresponding party labels. These form the training data used for the semi-supervised regression model.

I repeated the semi-supervised learning procedure for different values of M and J, where M is the number of Laplacian eigenvectors used as features and J is the number of labeled congress members used for training. For each pair (M, J), I built the Laplacian embedding using the first M eigenvectors, used the first J labeled members to train the regression model, predicted the labels of all 435 members, and then computed the classification accuracy.

TABLE 1. **Semi-Supervised Learning Accuracy for Different Values of M and J**

	J			
M	5	10	20	40
2	0.864	0.864	0.878	0.878
3	0.908	0.876	0.892	0.890
4	0.713	0.917	0.908	0.899
5	0.713	0.903	0.906	0.929
6	0.729	0.697	0.892	0.917

Rows correspond to the number of Laplacian eigenvector features M, and columns correspond to the number of labeled congress members J.

The results are summarized in Table 1. In general, the accuracy tends to improve as J increases, since more labeled examples are available for training. The results also show that values of M around 3 to 5 work best overall. However, when only a small number of labeled examples is available, using too many features can make the regression less stable. In that case, the model can start fitting patterns that do not generalize as well.

The best performance was obtained with $M = 5$ and $J = 40$, which gave an accuracy of 92.9%.

5. Summary and Conclusions

Spectral clustering and a semi-supervised regression method were applied to the 1984 U.S. House voting records dataset. First, a similarity graph was constructed from the voting patterns and the unnormalized graph Laplacian was used to perform spectral clustering. The second eigenvector (the Fiedler vector) was used to separate members into two clusters based on the sign of its entries.

By varying the Gaussian similarity parameter σ, the best clustering performance was achieved at $\sigma^* \approx 2.1$, giving a clustering accuracy of about 88%. This shows that the voting data contains a strong underlying two-group structure that corresponds closely to the political parties.

In the second part, Laplacian eigenvectors were used as graph-based features for a semi-supervised learning model. Even when only a small number of party labels were available, the regression model was able to correctly classify most members. The best performance was obtained using $M = 5$ eigenvector features and $J = 40$ labeled members, achieving an accuracy of approximately 92.9%.

Overall, these results demonstrate that spectral methods can effectively capture the structure of voting networks, and that combining graph features with a small amount of labeled data can significantly improve classification performance.

Acknowledgements

The author is thankful to Profesor Bamdad Hosseini for useful discussions about similarity function, graph Laplacian, spectral clustering principle, Laplacian embedding, and semi-supervised regression formulation.